Approaching the SkyWays capstone

How to think, build, present and answer questions the way a working engineer would.

Think
Find a real problem and the cause behind it.

Build
Build the smallest version that works end to end.

Prove
Show that it works on a bad day, with a test or a number.

Present
Tell the whole story clearly in ten minutes.

Answer
Handle the panel's questions, every member of the team.

Indian Institute of Technology Gandhinagar
Cohort 1 guided capstone session

October 2026
SkyWays Logistics is a fictional company

Where teams lost marks

Most of these mistakes happen before any code is written, and every one of them can be fixed.

Retelling the brief

"Drivers get delayed, so we built a chatbot."

Find the problem in the data first. p. 11

A tour of the tools

"We used LangGraph, ChromaDB, FastAPI and Streamlit."

Explain what each part decides. p. 15

A demo that breaks

"It was working on my laptop last night."

Rehearse it and keep a backup. p. 26

Numbers nobody can check

"Our accuracy is 92 percent."

Compare the model with a simple guess. p. 19

Running out of time

"Let me quickly show the demo," at minute nine.

Plan ten minutes and keep two spare. p. 25

One person answers everything

The same member answers every question.

Give every question an owner. p. 25

The questions the panel asks, and why

Each question checks one thing. If you know what it checks, you know what to show.

“Why did you pick this problem?”

It checks

Whether the problem came from evidence or was copied from the brief.

Show them

One number from the data and the cause behind it.

“What was your hypothesis?”

It checks

Whether you thought before you built.

Show them

The guess you started with and what the data said.

“How do you know it works?”

It checks

Whether it works beyond the one path you demoed.

Show them

A failure case you tested, or your model against a simple guess.

“Is it actually running?”

It checks

Whether what you claim matches what exists.

Show them

The honest status and the command that runs it.

“What does one run cost?”

It checks

Whether you have thought about running it every day.

Show them

Tokens per run, times the price, times runs per day.

“Who built which part?”

It checks

Whether every member did real work.

Show them

Each member explains their own part in detail.

What the panel wanted to see

Capstones that fail usually stop at level 1. The marks start at level 2, when you bring evidence.

Level 4 **Fixing the cause and proving it**	*"Holds now expire after ten minutes, and our test shows only one driver can get a slot."*	The panel asks what you would do next.
Level 3 **Finding the root cause**	*"A request holds a dock forever, because nothing makes it expire."*	The panel starts asking design questions.
Level 2 **Showing the pain with evidence**	*"At 11:00 on 4 August, two of the four docks were held by requests nobody had answered."*	The panel leans in.

Marks start above this line

Level 1 **Picking a feature**	*"We built slot search with LangChain and Streamlit."*	The panel hears a list of tools.
Level 0 **Retelling the brief**	*"SkyWays has delays at its warehouses."*	The panel hears nothing new.

Where the 100 marks come from

Only 35 of the 100 marks are for what you build. The other 65 are for how you think, prove and explain.

Think 35		Build 35		Prove	Present	Answer
15 Problem framing	20 Architecture	20 Track depth	15 Demonstration	10 Rigour	10 Communication	10 Questions
The pain with a number, and its root cause.	A clear picture, and one decision you can defend.	Real engineering on your track, even if not all of it shipped.	A demo that runs and shows a refusal.	A test or a metric anyone can rerun.	A clear ten-minute story.	Every member answers well.



In the last panel, the answers to questions separated teams more than any other column.

A team that builds well but answers badly loses the marks it earned.

Quick check: the first two hours

You receive the brief on day one. Where should the first two hours go?

A Choosing between LangGraph and CrewAI

B Designing the chat screen for drivers

C Reading the data and listing who is hurt, and how

D Setting up the slide template

Type A, B, C or D in the chat.

Answer: start with the data and the people

Until you know which problem is worth solving, a tool or a screen is only a guess.

C. Reading the data and listing who is hurt, and how

A The framework changes nothing about the problem, so it earns no marks for framing.

B A screen built first serves the brief's problem, which every other team is also building.

D Slides come last, because you do not know your story yet.

What the first two hours should produce

"At Jaipur on 4 August, four delayed trucks wanted the two docks that open at noon, while two docks at 11:00 sat locked by requests nobody had answered."

The driver waits at the gate.

The planner makes phone calls.

The warehouse loses dock time.

The ideal approach on one page

Start with the people and the data, write the problem down, then build the smallest working version.

Start here

1 Think

You do

Find who is hurt and show it with data. Ask why until you reach a cause you can change, then choose one fix.

You have

- `pain.md`
- `five_whys.md`
- `prd_lite.md`

2 Build

You do

Draw the parts and what each one decides. Build the thinnest version that runs from start to end.

You have

- `architecture.png`
- `working core`

3 Prove

You do

Test the bad days: two drivers at once, nobody replying. Compare any model with a simple guess.

You have

- `tests`
- `one number`

4 Present

You do

Write the README, make eight slides from your files, and rehearse three times.

You have

- `README.md`
- `slides`
- `demo script`

5 Answer

You do

List the likely questions, give each an owner, and swap parts so everyone can explain everything.

You have

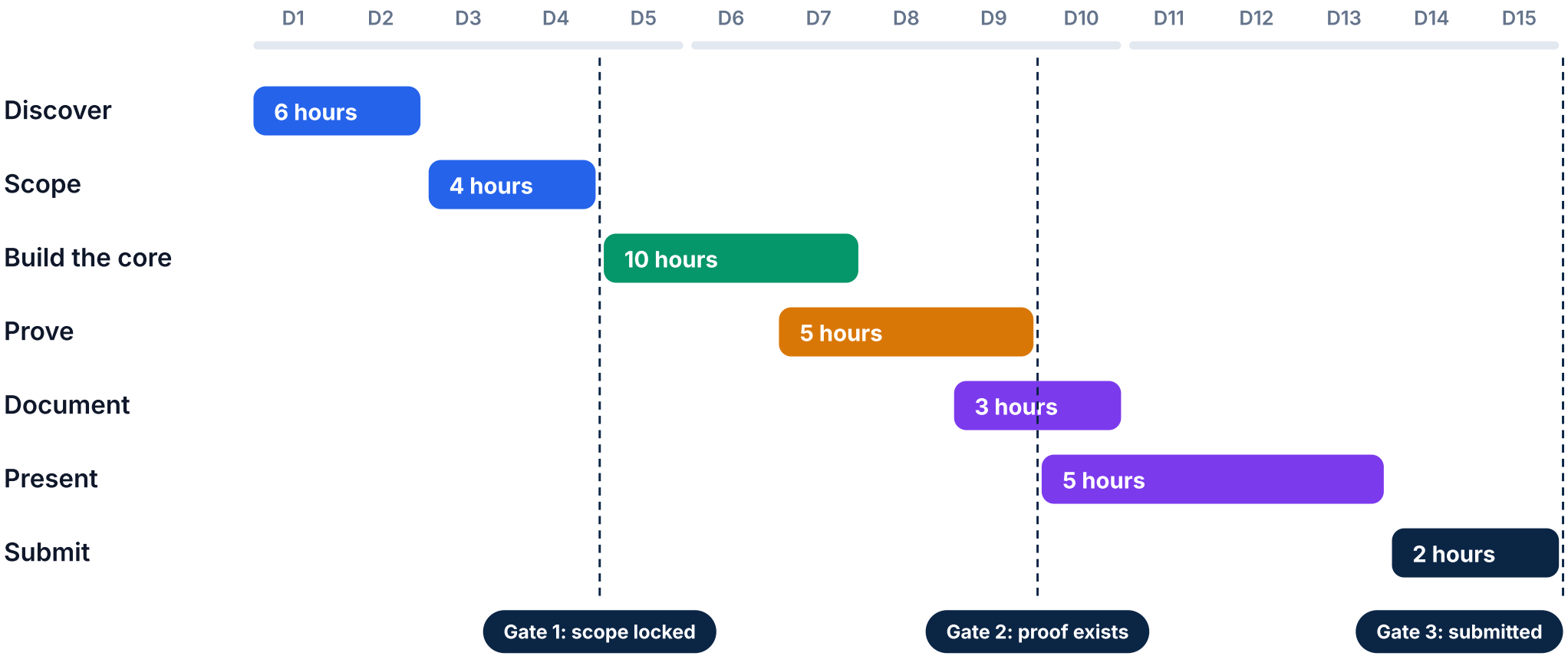
- `question list`

Repeat build and prove in small steps

Code starts only after the problem is chosen and written down.

How to spend the 35 hours

Four days to think and scope, five to build and prove, and a buffer week to polish and practise.



Ten of the 35 hours go to thinking before any code is written. A capstone that starts coding on day one usually presents the same problem as last time.

The same approach, done in two days

A forward deployed engineer runs the same nine steps in about thirteen hours, writing something down at each.

Day 1

about 6 hours

1 · 1 hour — Listen

Read the brief as a complaint from the operations team.

Who is hurt

2 · 2 hours — Look at the data

Find one real moment when things went wrong, with numbers.

The pain sentence

3 · 1 hour — Find the cause

Ask why until you reach something you can change.

Five whys

4 · 1 hour — Choose

Pick one fix that is worth it and fits the time.

One-page plan

5 · 1 hour — Sketch

Draw the parts and settle the one hard question.

One decision

Day 2

about 7 hours

6 · 3 hours — Build a thin version

Make one request work, from the driver's message to a held slot.

A path that runs

7 · 2 hours — Try to break it

Two drivers at once, a late truck, an unclear message.

Tests that pass

8 · 1 hour — Show a user

Demo it to a teammate playing the planner.

Their questions

9 · 1 hour — Write it up

How to run it, the key number, the limits.

README

Issue tree: break the problem into parts

Split one big question into smaller ones that do not overlap, then let the data pick the branch.

From McKinsey problem solving

What it is

A picture that breaks a big question into smaller ones that do not overlap and together cover everything.

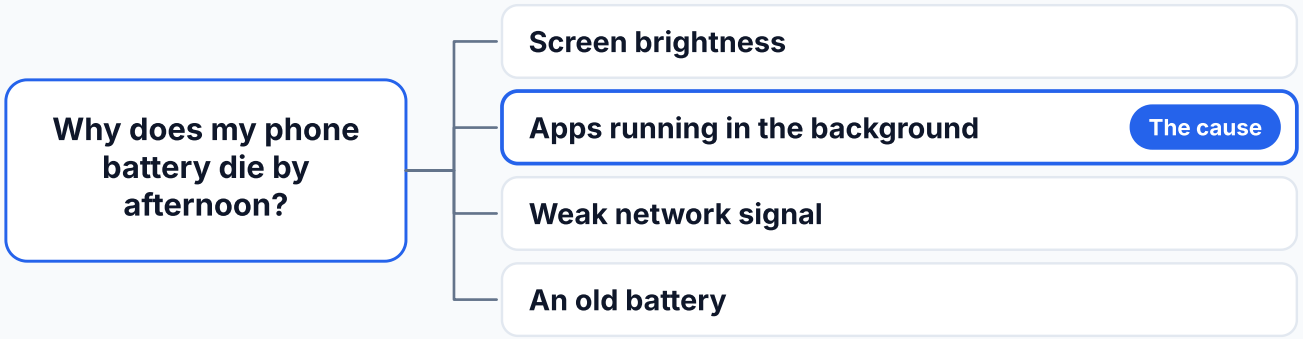
How to use it

1. Write the question at the top.
2. Split it into three or four branches that do not overlap.
3. Write a guess on each branch.
4. Check each guess with one piece of data, and drop the branches that fail.

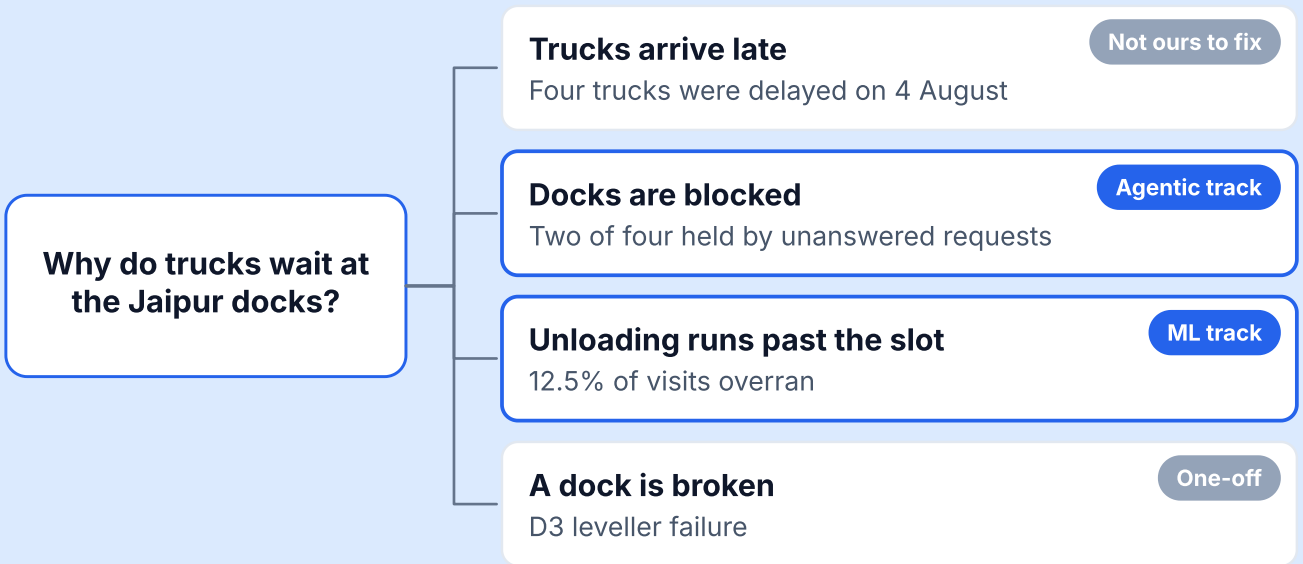
Use it when

you have a broad problem and do not yet know where to look.

Everyday example **the phone battery**



In SkyWays **the waiting trucks**



Mixed: issue trees are standard McKinsey practice; the branches and numbers are from the SkyWays data

Five Whys: dig down to the cause

Take one problem and keep asking why until you reach a cause you can change.

From the Toyota Production System

What it is

A chain of questions that starts at the symptom and ends at a cause you can actually fix.

How to use it

1. State the problem with a number.
2. Ask why, and answer with a fact from the data.
3. Repeat about five times.
4. Stop at a cause you can change, then read the chain upward with “so”.

Use it when

you have picked a branch of the issue tree and need its root cause.

Everyday example — a cold food delivery

The food arrived cold. **why** → It sat at the restaurant for fifteen minutes. **why** → It was ready before a rider arrived. **why** → Cooking starts as soon as the order is placed. **why** → **Cooking is not linked to when the rider will arrive.**

In SkyWays — the locked docks

Trucks wait while docks sit locked. **why** → Two docks are held by requests nobody answered. **why** → A request keeps its dock until someone replies. **why** → Nothing ever makes a request expire. **why** → **A booking has no expiry and no clear states.**

The fix: a hold that expires after ten minutes unless the warehouse confirms it.

Value and feasibility: choose what to build

Rate every idea on what it is worth and how likely you are to deliver it, then start top right.

From Gartner's AI use-case assessments

What it is

A two-by-two that places each idea by its value and by how easily you can deliver it in the time you have.

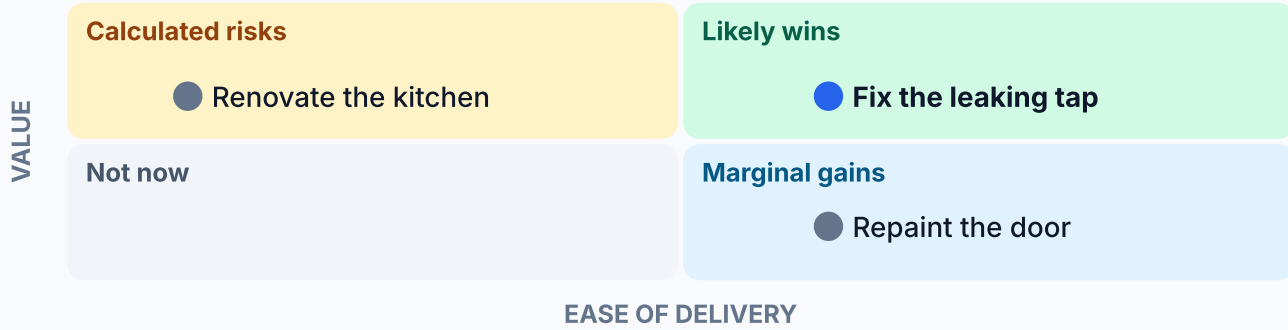
How to use it

1. List three to five ideas that attack the root cause.
2. Rate each on value: how much pain it removes.
3. Rate each on ease: can you deliver it in two weeks?
4. Start with the likely wins, and name the rest as future work.

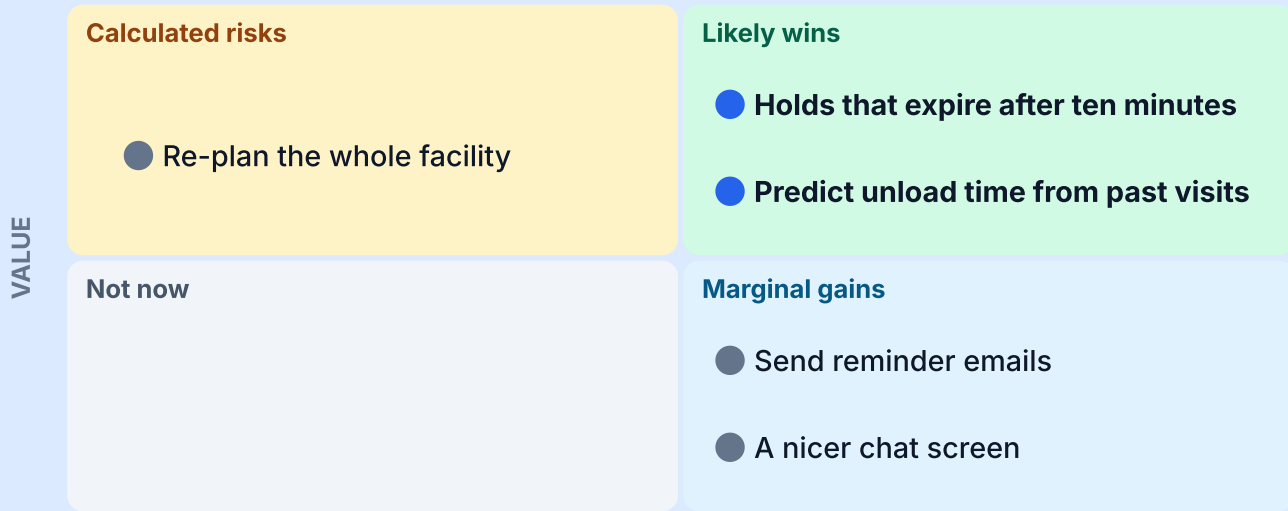
Use it when

you have more ideas than time, which is nearly always.

Everyday example **home repairs**



In SkyWays **the candidate fixes**



Mixed: Gartner sorts use cases into likely wins, calculated risks and marginal gains; the placements are ours

Analytics ascendancy: choose your track

Four kinds of answer, each harder and more useful than the last. The track you pick decides which one you build.

From Gartner's analytics model

What it is

Four questions in order of difficulty: what happened, why it happened, what will happen, and what to do about it.

How to use it

1. Answer the first two from the data before you build anything.
2. Choose ML if your build answers “what will happen”.
3. Choose agentic if it acts on “what should we do”.
4. Choose hybrid if a prediction feeds the action.

Use it when

you are deciding between the ML, agentic and hybrid tracks.

Everyday example a map app on your commute

What happened?	Why?	What will happen?	What should I do?
Traffic was heavy.	An accident on the highway.	You will reach 20 minutes late.	Take the service road.

In SkyWays the unload delays

What happened?	Why?	What will happen?	What to do?
12.5% of visits overran their slot.	Every standard slot is 60 minutes, whatever the load.	Predict how long each truck will take to unload.	Hold, rebook or escalate the slot.
Your evidence	Your root cause	ML track	Agentic track

Draw the system before you build it

One simple picture of the parts, and of what each part is allowed to decide.

From the C4 model by Simon Brown

What it is

A picture of the people and the parts of a system, with one line on what each part is responsible for.

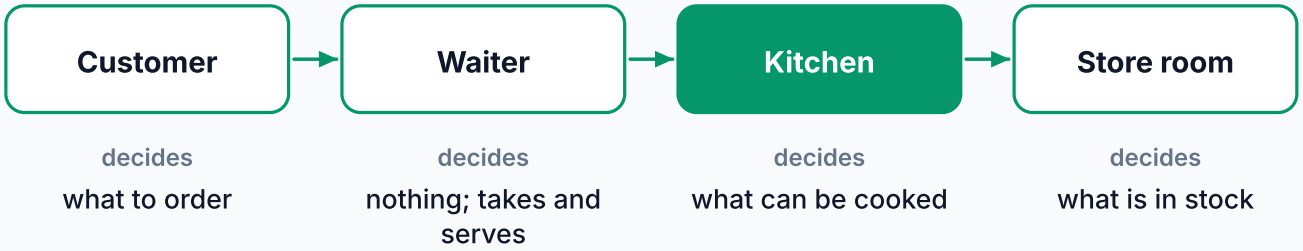
How to use it

1. Draw the people who use the system.
2. Draw the four or five main parts as boxes.
3. Join them with arrows that show who calls whom.
4. Under each box, write the one thing it decides.

Use it when

you are about to write code, and before every design review.

Everyday example — a restaurant



In SkyWays — the dock booking agent



The language model talks to the driver, but it never decides whether a slot is free or booked. The tools and the database do.

Build a thin version first

Get one request working from start to end before making any part of it good.

From the walking skeleton idea

What it is

The smallest version that works from start to end, which you then improve in steps that each still work.

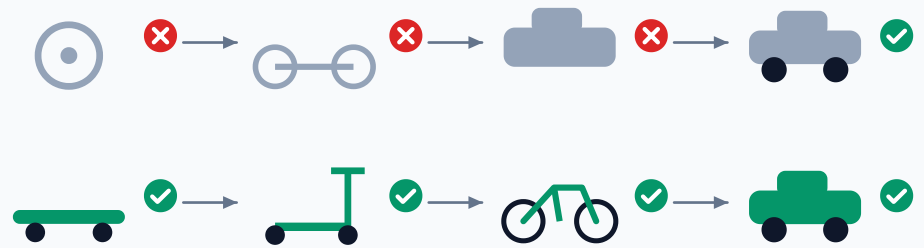
How to use it

1. Pick the one path the user cares about most.
2. Make it work end to end, even if it looks rough.
3. Add one improvement at a time.
4. Keep it working at the end of every day.

Use it when

you start building, and every time the deadline feels close.

Everyday example the skateboard before the car



After Henrik Kniberg

In SkyWays four versions, each one usable

Version 1 day 5	Version 2 day 6	Version 3 day 7	Version 4 day 8
A message in, one free slot shown, held, and a reply in the console.	The hold expires after ten minutes, and the slot frees up.	The warehouse is asked, and its answer reaches the driver.	Escalate when nothing fits, then add a simple screen.
Usable	Usable	Usable	Usable

Established: Alistair Cockburn's walking skeleton and Henrik Kniberg's MVP picture; the versions are ours

The key idea: a hold that expires

The SkyWays design is the same idea as a movie seat that is held while you pay.

The core design of the agentic track

What it is

A booking that is reserved for a short time, can belong to only one person, and is released automatically if nobody confirms it.

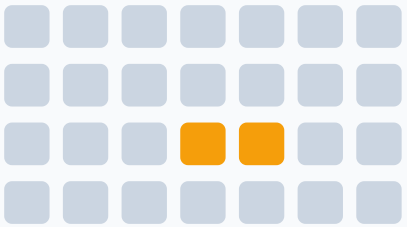
How to use it

1. Show only slots that are really free.
2. Hold the chosen slot for ten minutes.
3. Confirm it when the warehouse says yes.
4. Release it when the time runs out.

Use it when

the panel asks what stops a double booking, or what happens when nobody replies.

Everyday example — a movie ticket app



Two seats selected

Held for a few minutes while you pay

Paid: the seats are yours

Not paid: back on sale

Two people can never buy the same seat.

In SkyWays — the ten-minute dock hold

Shown

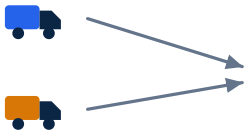
The driver sees the free slots.

Held, 10 minutes

Nobody else can take it.

Warehouse confirms

Released if no reply



Two drivers press at the same moment. The database accepts the first and refuses the second, so only one driver ever gets the slot.

Tests written as Given, When, Then

Write each test as a three-line story. The bad-day stories are the ones the panel will ask about.

From behaviour-driven development

What it is

A test written as a short story: the situation, the action, and the result you expect.

How to use it

1. Write the situation after Given.
2. Write the one action after When.
3. Write what must be true after Then.
4. Write the bad days first: two users at once, nobody replying, a request that does not fit.

Use it when

you are deciding what to test, or answering "how do you know it works?".

Everyday example — a cash machine

Given	My balance is Rs 500.
When	I try to withdraw Rs 700.
Then	The ATM refuses, and my balance stays Rs 500.

In SkyWays — three bad days to test

Given	Two drivers want D2 at 12:00.
When	Both ask at the same moment.
Then	Exactly one gets the hold.

Given	A hold was made ten minutes ago.
When	Nobody has confirmed it.
Then	The slot shows as free again.

Given	A truck needs 75 minutes.
When	Only 60-minute slots are free.
Then	It is not offered one, and it is escalated.

Established: Given, When, Then comes from Dan North's behaviour-driven development; the cases are from the SkyWays data

ML track: beat a simple guess

A model is only worth showing if it beats the obvious guess on data it has never seen.

Standard practice for evaluating models

What it is

Compare your model with the simplest sensible guess, on data that comes later in time than the data you trained on.

How to use it

1. Write down the simple guess first.
2. Split by time: train on earlier data, test on later data.
3. Report the error in units the user feels, such as minutes.
4. Show one case it gets right and one it gets wrong.

Use it when

you present any model, and before you quote any accuracy number.

Everyday example the weather forecaster

The simple guess

"Tomorrow will be like today."

beat it?

The forecaster's model

Useful only if it is right more often than the simple guess.

In SkyWays unload time at the docks

Average error in minutes, tested on July visits

Simple guess: the usual time for the dock type — 11.5

The model — 7.8

101 of 120

July overruns caught when slots were sized from the model.

Jan | Feb | Mar | Apr | May | Jun | **Jul**

Trained on the months before July, tested on July, so the model never sees the future.

Using AI tools the right way

Treat the assistant as a fast helper whose work you check. The decisions and the numbers stay yours.

Where it helps

Understand the domain

Ask it to play the warehouse planner and to question you about the problem.

Find references

Ask how ticket and ride apps handle holds, then open and read every source it names.

Draft code you have specified

Give it the function, the inputs and your tests, and read every line it returns.

Invent awkward test cases

Ask for messy driver messages, such as Hinglish or half-finished sentences.

Play the panel

Ask for the ten hardest questions about your design, one at a time.

Where it hurts you

Choosing the problem for you

It only knows the brief, so it repeats the brief.

Numbers you did not compute

The panel asks where each number came from.

Code you cannot explain

You will be asked to explain it line by line.

Slide text pasted from a chat

The panel recognises the generic tone at once.

References you never opened

Some of them will not exist.

Four prompts worth copying

Each prompt gives the assistant a role, a task and a format, and each answer gets checked before you use it.

Think To understand the problem

You are the operations planner at a warehouse with four standard docks. I will describe a delay. Ask me ten questions you would need answered before you could help.

Then: Keep the questions the data can answer.

Think To find references

List three ways ticket or ride-hailing apps stop two people from booking the same seat. For each one, name a public source I can read.

Then: Open every source before you use it.

Build To draft code

Here is the function name, its inputs and three Given, When, Then tests. Write the function and explain each line in one sentence.

Then: Run the tests and read every line.

Answer To rehearse the panel

You are a strict capstone panel. Here is our system picture. Ask the five hardest questions one at a time, and tell me what a strong answer contains.

Then: Answer without notes, then compare.

After the build: what to finish, in order

Plan the last days as carefully as the build, because they decide how the work is seen.

1

Freeze the features

Stop adding features two days before the panel.

2

Write the README

A stranger should be able to run it with three commands.

3

Record a backup demo

A two-minute video, in case the live run fails.

4

Make slides from your files

Every number on a slide comes from the repository.

5

Rehearse three times

Use a timer, and cut words when you run long.

6

Swap parts

Each member explains a teammate's part out loud.

7

List the likely questions

Ten questions, each with an owner and a short answer.

8

Run it from a fresh copy

Clone the repository on another laptop and run it once.

Pyramid Principle and SCQA: lead with the answer

Say the answer first and support it underneath. SCQA turns your first minute into four sentences.

From Barbara Minto at McKinsey

What it is

Give the answer first, then the reasons, then the evidence. SCQA is a four-step way to open: Situation, Complication, Question, Answer.

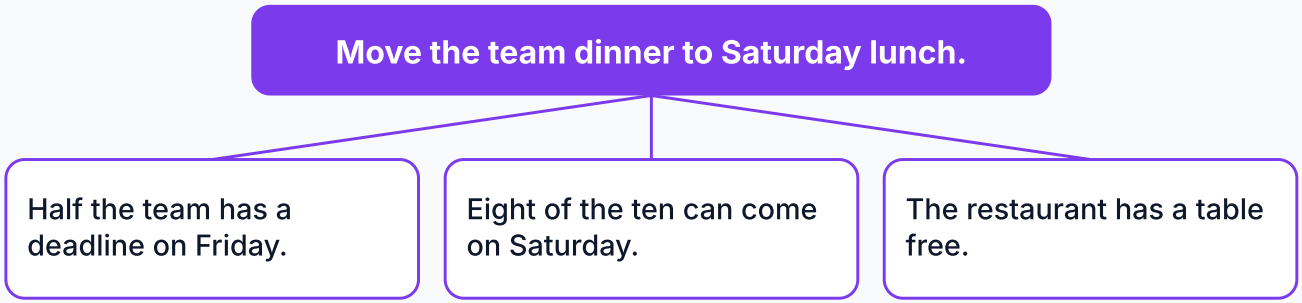
How to use it

1. Situation: what everyone already agrees on.
2. Complication: what went wrong, with a number.
3. Question: what the listener now wants to know.
4. Answer: your solution in one sentence, then the reasons.

Use it when

you plan the first minute of the presentation, or any answer to a senior person.

Everyday example **moving a team dinner**



In SkyWays **the first minute of your talk**

S	Situation	SkyWays books dock slots for trucks at its Jaipur warehouse.
C	Complication	On 4 August, two of four docks sat locked by requests nobody answered, while four delayed trucks waited.
Q	Question	How do we stop unanswered requests from blocking docks?
A	Answer	**Hold a slot for ten minutes, release it if nobody confirms, and let only one driver have it.**

The ten-minute presentation, slide by slide

Eight slides, each with a title that states its point, telling the story from the problem to the proof.

1 The problem

Four delayed trucks wanted two free docks while two others sat locked.

On screen: The dock grid at 11:00

A — 1:00

2 The root cause

Requests never expire, so they block docks.

On screen: The five whys

A — 1:00

3 The system

The agent talks, the tools decide, and the database records.

On screen: The system picture

B — 1:00

4 The key decision

A held slot expires after ten minutes unless it is confirmed.

On screen: Three options

B — 1:00

5 Live demo

A delay becomes a held slot, and an impossible request is escalated.

On screen: The running system

C D — 3:00

6 The proof

Four drivers try for one slot at once, and exactly one gets it.

On screen: The test result

D — 1:00

7 The limits

What it cannot do yet, and what the full version would need.

On screen: Three limits

D — 0:30

8 Next step

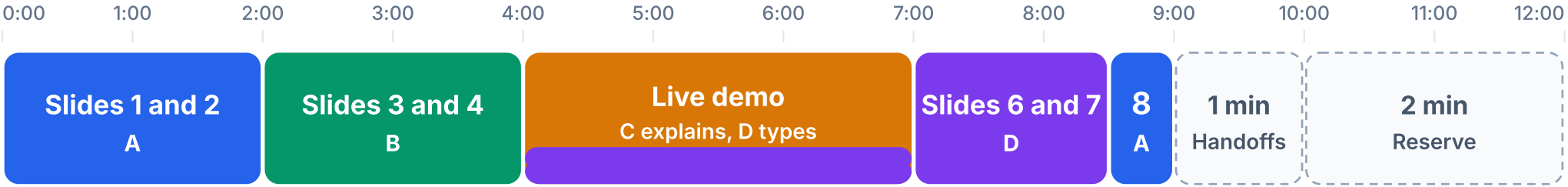
What we would build next, and roughly how long it would take.

On screen: One next step

A — 0:30

Splitting the presentation between three or four people

Each person presents the part they built, so every question has an obvious owner.



A Problem owner

Presents
Slides 1, 2 and 8

Built
The pain and the five whys

Answers
Why this problem?

B Architect

Presents
Slides 3 and 4

Built
The system picture and the decision

Answers
How does it work?

C Builder

Presents
Explains the demo

Built
The agent and its tools

Answers
What if it breaks?

D Tester

Presents
Types the demo, slides 6 and 7

Built
The tests and the results

Answers
How do you know?

In a group of three, B also tests, so B types the demo and presents slides 6 and 7.

Plan ten minutes and keep two in reserve for a slow demo or a late question.

Running the demo without surprises

Treat the demo like a short rehearsed play: the same start, the same script, and a backup ready.

1

Reset the data

Restore the same starting database before every run.

2

Follow a script

Type the exact messages you rehearsed, in the same order.

3

Split the roles

One person types while another explains what the panel sees.

4

Show a refusal

Include one request the system should turn down.

5

Keep a backup

If anything breaks, switch to the recording within ten seconds.

The demo the panel remembers

A real driver message → A held slot → The warehouse asked → Four drivers, one slot → A request refused

Quick check: a question you cannot answer

A panelist asks something you do not know. What do you do?

A Give your best guess, with a number



B Say you do not know, and how you would find out



C Pass it to the strongest member of the team

D Move quickly to a part you know well

Type A, B, C or D in the chat.

Answer: say what you would check

An honest “I don't know, and here is how I would find out” keeps the panel's trust.

B. Say you do not know, and how you would find out

A A guessed number loses trust fast, because the panel asks where it came from.

C The panel then asks you a separate question, and the gap shows.

D Panels notice a dodged question and usually ask it again.

What a good “I don't know” sounds like

“We haven't measured that yet. I would run the replay with fifty drivers instead of four and time each hold. I can send you the result after the session.”

It admits the gap
in the first sentence.

It names the check
you would run.

It offers a next step
and keeps your credibility.

PREP: a simple shape for every answer

Answer first, then back it up with one reason and one fact from your work.

A common public-speaking method

What it is

Answer in four short moves: your point, the reason, an example, and your point again.

How to use it

1. Point: answer the question in your first sentence.
2. Reason: give the one reason that matters most.
3. Example: a fact, a test or a number from your work.
4. Point: repeat the answer in one line, then stop.

Use it when

you answer any panel question, and in every interview.

Everyday example — metro or cab to the airport

P	Point	Take the metro.
R	Reason	Traffic cannot delay it.
E	Example	Last Friday the cab took 90 minutes, the metro 40.
P	Point	So the metro is safer for an evening flight.

In SkyWays — a panel question

"What stops a double booking?"

P	Point	The database allows only one active booking per slot.
R	Reason	A unique rule on the slot refuses the second booking, whatever the code does.
E	Example	In our replay, four drivers tried for the same slot at once, and three were refused.
P	Point	So even if two drivers press together, only one gets the slot.

Hard questions and good answers

These come up in almost every panel. Each good answer rests on a number or on something the panel can check.

“Isn't this just a chatbot?”

The chat is only how the driver talks to it. What matters is the slot underneath: held, expiring and never double-booked.

Backed by the system picture

“Why use an LLM? A form would do.”

21 of the 148 real driver messages needed a follow-up question, like “Late hounga thoda.” A form cannot ask one.

Backed by the labelled messages

“What if it says confirmed too early?”

It can say confirmed only after the warehouse replies, and one of our tests checks exactly that.

Backed by the test case

“Your model is still 7.8 minutes off.”

The simple guess was 11.5 minutes off, and the model would have caught 101 of 120 overruns in July.

Backed by the evaluation

“What does one conversation cost?”

About six model calls, roughly 2 US cents at the prices we assumed, so under USD 15 a month at 25 a day.

Backed by your cost sum

“Who built the agent?”

asked to the quietest member

Name the teammate who built it, then explain how it works yourself, in your own words.

Backed by every member

Your next 48 hours

Five files, finished before any agent code exists, each with an owner and a check you can make by opening it.

1	`queries.sql` `pain.md`	A	Every number in the pain sentence comes from the data.
2	`five_whys.md`	A with everyone	Every why is backed by a fact.
3	`prd_lite.md`	A B	One problem, one fix, and a list of what you will not build.
4	`architecture.png` `adr_002_hold.md`	B	Every box says what it is allowed to decide.
5	`run_of_show.md`	A B C D	Speakers and question owners named, first rehearsal booked.

Where the brief goes deeper

Think and scope
Brief pages 12 to 20

Design
Brief pages 21 to 33

Present and defend
Brief pages 38 to 43

The whole approach on one page

Fifteen habits, in the order you will use them.

Think

- Find who is hurt, with data.
- Ask why until you can change it.
- Choose one fix.

Build

- Draw what each part decides.
- Build a thin version first.
- Keep it running every day.

Prove

- Test the bad days.
- Beat a simple guess.
- Keep proof in files.

Present

- Lead with the answer.
- Present what you built.
- Rehearse three times.

Answer

- Give every question an owner.
- Point, reason, example, point.
- Say what you would check.